Chủ đề 04: Kiến trúc Clean Architecture & Dependency Injection (DI)

TRƯỜNG ĐẠI HỌC BÁCH KHOA — ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần 01 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052

GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Kiến Trúc 3 Tầng & Nguyên Lý DIP
   - Phân chia 3 tầng theo Clean Architecture
   - Nguyên lý Đảo ngược Phụ thuộc (DIP)
2. Phần II: Dependency Injection (DI) & Khởi Tạo Phụ Thuộc
   - Service Locator với get_it
   - Khởi tạo hệ thống & MultiBlocProvider
3. Phần III: Demo Thực Tế & Unit Test Tầng Domain
   - Ứng dụng Demo REST API tối giản

- Unit Test Tầng Domain với Mocktail
- Tài liệu tham khảo

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Nghiên cứu kiến trúc 3 tầng, thiết kế Domain, cấu hình DI (get_it), viết UI BLoC và Unit Test	100%

Phần I: Kiến Trúc 3 Tầng & Nguyên Lý DIP

Phân chia 3 tầng theo Clean Architecture

- **Presentation (Giao diện):**
  1. `PostPage` (UI hiển thị trắng đen tối giản) & `PostBloc` (quản lý trạng thái).
  2. Chỉ nhận dữ liệu từ Domain, **không biết dữ liệu lấy từ đâu**.
- **Domain (Nghiệp vụ - Pure Dart 100%):**
  1. `Post` (Entity), `GetPostsUseCase` (hành động nghiệp vụ), `PostRepository` (Hợp đồng Interface).
  2. Không dính líu Flutter hay HTTP, giúp chạy Unit Test độc lập tính bằng mili-giây.
- **Data (Nguồn dữ liệu):**
  1. `PostRemoteDataSource` (gọi REST API), `PostModel` (parse JSON), `PostRepositoryImpl` (thực hiện hợp đồng).

Nguyên lý Đảo ngược Phụ thuộc (DIP)

- **Bản chất của DIP:**
  1. Module cấp cao (`Presentation`) và module cấp thấp (`Data`) cùng phụ thuộc vào **Interface trừu tượng** ở giữa (`Domain`).

Nguyên lý Đảo ngược Phụ thuộc (DIP)

Minh chứng trong mã nguồn dự án

1. Tầng `Domain` đặt ra hợp đồng: `abstract class PostRepository`.
2. Tầng `Data` hiện thực hợp đồng: `class PostRepositoryImpl implements PostRepository`.
3. Tầng `Presentation` (BLoC) chỉ gọi `GetPostsUseCase`, không bao giờ import tầng Data.

→ Khi đổi nguồn dữ liệu (REST API ⟶ SQLite hoặc Mock), tầng UI giữ nguyên 100%.

Phần II: Dependency Injection (DI) & Khởi Tạo Phụ Thuộc

Service Locator với get_it

- **Tại sao cần DI?**
  1. Không để các lớp tự khởi tạo (`new`) lẫn nhau, giúp code lỏng lẻo (loose coupling) và dễ test.

- **Hai cơ chế đăng ký trong file `injection_container.dart`:**
  1. `registerLazySingleton`: Tạo duy nhất 1 bản thể khi được gọi lần đầu (tiết kiệm bộ nhớ cho `HttpClient`, `DataSource`, `Repository`, `UseCase`).
  2. `registerFactory`: Tạo mới 1 bản thể mỗi lần yêu cầu (dùng cho `PostBloc` gắn theo vòng đời màn hình).

Khởi tạo hệ thống & MultiBlocProvider

- **`WidgetsFlutterBinding.ensureInitialized()`:**
  1. Đảm bảo Flutter Engine và kênh giao tiếp nền tảng sẵn sàng trước khi nạp service bất đồng bộ (`async`) trong `main()`.

- **`MultiBlocProvider`:**
  1. Lấy instance `PostBloc` từ `get_it` và đưa vào cây widget của Flutter.
  2. Tự động gọi `close()` giải phóng bộ nhớ khi màn hình bị hủy, chống rò rỉ RAM (Memory Leak).

Phần III: Demo Thực Tế & Unit Test Tầng Domain

Ứng dụng Demo REST API tối giản

- **Nguồn dữ liệu:** JSONPlaceholder (`https://jsonplaceholder.typicode.com/posts`).
- **Giao diện:** Phong cách trắng đen tối giản, tập trung hoàn toàn vào việc thể hiện luồng dữ liệu 3 tầng:
  1. Đang tải $\longrightarrow$ Vòng xoay tiến trình (`Loading`).
  2. Tải xong $\longrightarrow$ Danh sách bài viết bài bản (`Loaded`).
  3. Mất mạng $\longrightarrow$ Thông báo lỗi & nút Thử lại (`Error`).
- **Bằng chứng tách biệt:** File `post_page.dart` tuyệt đối không có dòng import `data/`.

Unit Test Tầng Domain với Mocktail

- **Kỹ thuật Mocking:**
  1. Dùng thư viện `mocktail` tạo `MockPostRepository extends Mock implements PostRepository`.
  2. Đóng vai “người cung cấp dữ liệu giả”, test độc lập UseCase không cần mạng Internet.

Unit Test Tầng Domain với Mocktail

Kết quả kiểm thử tự động (flutter test)

- Test Case 1: Lấy dữ liệu thành công $\longrightarrow$ UseCase trả về `Right(List<Post>)`.
- Test Case 2: Máy chủ phản hồi lỗi $\longrightarrow$ UseCase trả về `Left(ServerFailure)`.
- **Kết quả thực tế:** 5/5 test cases pass 100% trong thời gian < 1 giây.

Tài liệu tham khảo

1. Robert C. Martin (2017), *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
2. Flutter Official Documentation: Architectural Overview & Testing.
3. BLoC Library Documentation: https://bloclibrary.dev
4. Mã nguồn hoàn chỉnh: https://github.com/hthienloc/dut-cross-platform-programming